DS603: Data Structures and Algorithms in Python

Laboratory 07

Advanced Object-Oriented Programming in Python

Concepts, Design Patterns, Examples, and Practical Problems

Purpose of this laboratory. This laboratory moves beyond writing isolated classes. The goal is to understand how Python classes cooperate inside a larger software design: shared class state, alternative constructors, inheritance hierarchies, method resolution order (MRO), cooperative `super()`, composition, polymorphism, and Python's data-model methods.

Course	DS603 – Data Structures and Algorithms in Python
Laboratory	07 – Advanced Object-Oriented Programming in Python
Part	Part 2 of the OOP sequence
Issued	September 30, 2026
Due	October 13, 2026
Submission	One Jupyter Notebook: `<StudentID>_Lab07.ipynb`

How to use this handout. Read the concept notes before attempting the problems. For every concept, first understand *what Python does*, then understand *why the design is useful*, and finally trace a small example by hand. The practical problems deliberately combine several concepts.

Contents

1 Laboratory Overview and Learning Outcomes **4**

1.1 Learning outcomes . . . 4

1.2 What is new in Lab 07? . . . 4

1.3 A useful mental model . . . 4

2 Concept 1: Instance State vs Class State **5**

2.1 Instance variables . . . 5

2.2 Class variables . . . 5

2.3 Attribute lookup and shadowing . . . 5

2.4 Common mistake . . . 5

3 Concept 2: Instance Methods, Class Methods, and Static Methods **6**

3.1 The three method types . . . 6

3.2 Why does `self` exist? . . . 6

3.3 Why does `cls` exist? . . . 6

3.4 Static methods . . . 6

4 Concept 3: Alternative Constructors **7**

5 Concept 4: Inheritance – IS-A Relationships **7**

5.1 What inheritance means . . . 7

5.2 Types of inheritance . . . 8

5.3 Method overriding . . . 8

6 Concept 5: `super()` and Cooperative Inheritance **8**

6.1 What `super()` actually means . . . 8

6.2 Why explicit parent calls can be dangerous . . . 9

7 Concept 6: Method Resolution Order (MRO) and C3 Linearization **9**

7.1 Why MRO is necessary . . . 9

7.2 C3 MRO . . . 9

7.3 MRO and `super()` are connected . . . 9

8 Concept 7: Polymorphism and Simulated Overloading **10**

8.1 Polymorphism . . . 10

8.2 Method overriding vs method overloading . . . 10

9 Concept 8: Composition – HAS-A Relationships **10**

9.1 Inheritance vs composition . . . 10

10 Concept 9: Python Data-Model (Dunder) Methods **11**

10.1 A container example 11

10.2 How $_{l}en_{and}$ 12

10.3 $_{r}epr_{vs}$ 12

11 Concept 10: Encapsulation and Abstraction in the Capstone **12**

11.1 Encapsulation 12

11.2 Abstraction with `ABC` 12

12 Concept 11: Putting the Concepts Together **12**

13 Assignment Problems (90-Minute Practical Session) **14**

13.1 Problem 01: Campus Student Registry & Class-Level State Engine 14

13.2 Problem 02: Corporate Payroll Hierarchy (Multilevel & Hierarchical Inheritance) . . . 16

13.3 Problem 03: Smart-Home Device Fabric (Multiple Inheritance & MRO) 18

13.4 Problem 04: Smart Shopping Cart: A Native-Feeling Container (Composition & Dunder Methods) 20

13.5 Problem 05: Mini Core-Banking Platform (Capstone) 22

Laboratory Overview and Learning Outcomes

Lab 06 introduced classes, objects, encapsulation, abstraction, single inheritance, polymorphism, and basic operator overloading. Lab 07 extends those ideas to systems in which several classes collaborate. The emphasis is not merely on syntax; students should be able to explain the design decisions behind the code.

Learning outcomes

By the end of this laboratory, a student should be able to:

1. distinguish **instance state** from **class-level state**;
2. explain how `self` and `cls` are supplied to methods;
3. write and use `@classmethod` and `@staticmethod`;
4. implement alternative constructors using `cls(...)`;
5. explain class-variable shadowing and Python's attribute lookup;
6. distinguish single, multilevel, hierarchical, multiple, and hybrid inheritance;
7. explain method overriding and polymorphic dispatch;
8. trace Python's **C3 Method Resolution Order (MRO)**;
9. use cooperative `super()` correctly in multiple-inheritance hierarchies;
10. distinguish **inheritance (IS-A)** from **composition (HAS-A)**;
11. implement Python data-model methods such as `__len__`, `__getitem__`, `__contains__`, `__iter__`, `__bool__`, `__call__`, `__repr__`, and `__str__`; and
12. combine abstraction, encapsulation, inheritance, composition, and polymorphism in a small software system.

What is new in Lab 07?

Lab 06 foundation	Classes, `self`, `__init__`, properties, encapsulation, abstraction, single inheritance, overriding, polymorphism, basic comparison/arithmetic dunders.
Lab 07 focus	Class variables and shadowing; `cls`; class methods and alternative constructors; static methods; multilevel/hierarchical/multiple inheritance; C3 MRO; cooperative `super()`; composition; simulated overloading; container/callable dunders; `isinstance` and `issubclass`.

A useful mental model

A Python class should be viewed in two ways:

1. as a **blueprint/type** from which objects are created; and
2. as an **object in its own right**, with its own namespace containing class attributes and methods.

This second viewpoint is essential for understanding class variables, class methods, inheritance, MRO, and descriptors such as properties.

Concept 1: Instance State vs Class State

Instance variables

An **instance variable** belongs to one particular object. It is normally created using `self.attribute` inside `__init__`. Two objects of the same class can therefore hold different values.

> **Example.** If `Student("Asha", 8.5)` and `Student("Ravi", 9.2)` are created, each object stores its own `name` and `cgpa`. Changing Asha's CGPA does not change Ravi's CGPA.

Class variables

A **class variable** is defined in the class body. It belongs to the class namespace and is normally shared by all instances that do not shadow it. Class variables are appropriate for information such as a university name, a company name, or a global object counter.

```
class Student:
    university = "DS_University"      # class variable
    student_count = 0                 # class variable

    def __init__(self, name: str, cgpa: float) -> None:
        self.name = name              # instance variable
        self.cgpa = cgpa              # instance variable
        Student.student_count += 1
```

Here, every student has a separate `name` and `cgpa`, but they can all observe the same `Student.university` and `Student.student_count`.

Attribute lookup and shadowing

When Python evaluates `obj.x`, it performs attribute lookup. For ordinary instance attributes, the important intuition is:

`obj.__dict__` → class namespace → parent classes following the MRO.

Suppose `Student.university = "DS_University"`. If we execute:

```
asha = Student("Asha", 8.5)
asha.university = "Legacy_Institute"
```

Python does *not* change `Student.university`. Instead, it creates an instance attribute `asha.university`. This is called **shadowing**: the instance-level value hides the class-level value for that object.

> **Remember:** `obj.x = value` normally writes to the object. `ClassName.x = value` writes to the class. Therefore, use the class or a class method when the intention is to update shared state.

Common mistake

A frequent error is assuming that changing a class variable through one instance changes the shared variable:

```
class Employee:
    company = "TechCorp"

```

```
e1 = Employee()
e2 = Employee()
e1.company = "StartUpX"
print(e2.company)       # TechCorp
print(Employee.company) # TechCorp
```

The assignment created `e1.company`; it did not modify `Employee.company`.

Concept 2: Instance Methods, Class Methods, and Static Methods

The three method types

The following table should be memorized conceptually, not merely syntactically.

Method	First argument	Main purpose
Instance method	`self`	Operates on one object and its state.
Class method	`cls`	Operates on class-level state; often used for alternative constructors.
Static method	None	Utility logic that belongs conceptually to the class but does not need an object or class.

Why does `self` exist?

When we write:

```
student.show_info()
```

Python conceptually supplies the object to the method, so a method such as `show_info(self)` can access the state of that particular student.

Why does `cls` exist?

A class method receives the class on which it was called. This matters when the method should construct or modify class-level state.

```
class Student:
    university = "DS_University"

    def __init__(self, name: str):
        self.name = name

    @classmethod
    def change_university(cls, name: str) -> None:
        cls.university = name
```

Calling `Student.change_university("Global Tech")` changes the class-level value. Using `cls` rather than hard-coding `Student` also makes the method more inheritance-friendly.

Static methods

A static method does not receive `self` or `cls`. It is appropriate for a helper that logically belongs to the class but does not need object/class state.

```python
class Student:
    @staticmethod
    def is_valid_cgpa(cgpa: float) -> bool:
        return 0.0 <= cgpa <= 10.0
```

The method is related to students, but the validation rule itself does not depend on a particular student object or on the class's current state.

Concept 3: Alternative Constructors

Python has one ordinary `__init__` method per class; it does not provide Java-style constructor overloading by defining multiple `__init__` methods with different parameter lists. A common Python design is to provide **alternative constructors** using class methods.

```python
class Customer:
    def __init__(self, name: str, account_type: str):
        self.name = name
        self.account_type = account_type

    @classmethod
    def from_name(cls, name: str) -> "Customer":
        return cls(name, "Savings")

    @classmethod
    def from_dict(cls, data: dict) -> "Customer":
        return cls(data["name"], data.get("account_type", "Checking"))
```

There are now three construction paths:

- `Customer("Alice", "Premium")` – ordinary constructor;
- `Customer.from_name("Bob")` – alternative constructor from a name; and
- `Customer.from_dict({...})` – alternative constructor from structured data.

Why use `cls(...)` instead of `Customer(...)`? If a subclass inherits `from_dict()`, using `cls(...)` allows the subclass to be constructed. Hard-coding `Customer(...)` would force the base class to be returned.

Concept 4: Inheritance – IS-A Relationships

What inheritance means

Inheritance allows a child class to reuse and extend a parent class. The key design question is whether the child really represents an **IS-A** relationship with the parent.

For example, a `Manager` is an `Employee`, so inheritance is natural. A `Car` is not an `Engine`; a car *has an* engine, so composition is more appropriate.

Types of inheritance

Type	Meaning
Single	One parent and one child.
Multilevel	A chain such as `Employee -> Manager -> Director`.
Hierarchical	Multiple children share one parent, e.g. `Engineer` and `Intern` derive from `Employee`.
Multiple	One child has multiple direct parents, e.g. `Doorbell(Camera, Speaker)`.
Hybrid	A combination of two or more inheritance patterns.

Method overriding

If a child defines a method with the same name as a parent method, the child implementation **overrides** the parent implementation.

```
class Account:
    def account_type(self) -> str:
        return "General Account"

class SavingsAccount(Account):
    def account_type(self) -> str:
        return "Savings Account"
```

The same call, `account.account_type()`, can produce different behavior depending on the actual object. This is the basis of **runtime polymorphism**.

Concept 5: `super()` and Cooperative Inheritance

What `super()` actually means

A common beginner explanation says that `super()` means "call my parent." That explanation is incomplete and becomes misleading in multiple inheritance.

A better rule is:

> **`super()` means "continue method lookup from the next class in the MRO."**

For simple single inheritance this often looks like calling the parent, which is why the simplified explanation appears to work. In multiple inheritance, however, `super()` is what allows all participating classes to cooperate.

```
class Employee:
    def describe(self) -> str:
        return "Employee"

class Manager(Employee):
    def describe(self) -> str:
        return super().describe() + " -> Manager"
```

The child extends the parent's behavior rather than copying it.

Why explicit parent calls can be dangerous

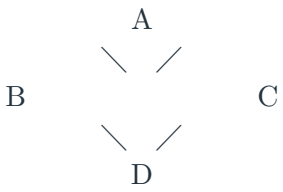
In a multiple-inheritance hierarchy, writing `Parent.method(self)` manually bypasses the MRO. It can cause a shared base class to execute more than once.

Therefore, when a class is designed to participate in a cooperative multiple-inheritance hierarchy, each implementation should generally call `super()` rather than naming a specific parent.

Concept 6: Method Resolution Order (MRO) and C3 Linearization

Why MRO is necessary

Consider the diamond:



Here both `B` and `C` inherit from `A`, while `D` inherits from both `B` and `C`. If `D` asks for a method that is defined in several classes, Python needs one deterministic order in which to search.

C3 MRO

Python computes an MRO using **C3 linearization**. The result respects important consistency rules, including:

- a class appears before its parents;
- the declared left-to-right base-class order is respected; and
- the ordering remains consistent with the parent classes' own MROs.

For:

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

print([c.__name__ for c in D.mro()])
```

we obtain:

```
['D', 'B', 'C', 'A', 'object']
```

MRO and `super()` are connected

For an object of type `D`, `super()` follows the computed MRO. Thus, if `D.method()` calls `super().method()`, the next implementation is `B.method()`, then `C.method()`, then `A.method()` when each class cooperates.

> **Viva rule:** If asked "Who is the parent of B in `super()`?" do not answer only "A." The precise answer is that `super()` continues lookup from the current class in the MRO of the instance's type.

Concept 7: Polymorphism and Simulated Overloading

Polymorphism

Polymorphism means that code can use a common interface while different object types provide different implementations.

For example, a payroll system can keep a list containing `Employee`, `Manager`, `Engineer`, and `Intern` objects and simply call:

```
for employee in employees:
    print(employee.monthly_pay())
```

The driver does not need a chain of `if isinstance(...)` statements to decide how each salary is calculated. The object's method dispatch handles that decision.

Method overriding vs method overloading

These terms must not be confused.

Overriding	A subclass supplies a new implementation of an inherited method with the same name.
Overloading	Multiple signatures for the same operation. Python does not support Java/C++-style signature-based method overloading directly; it is commonly simulated using default arguments or `*args`/`**kwargs`.

```
class Printer:
    def show(self, *args) -> str:
        if len(args) == 1:
            return f"one value: {args[0]}"
        if len(args) == 2:
            return f"two values: {args[0]}, {args[1]}"
        raise TypeError("Unsupported arguments")
```

Concept 8: Composition – HAS-A Relationships

Inheritance vs composition

Use inheritance when the relationship is genuinely **IS-A**. Use composition when an object **HAS-A** another object.

Design	Example	Meaning
Inheritance	`Manager(Employee)`	A manager is an employee.
Composition	`Car` stores an `Engine`	A car has an engine.
Composition	`Customer` stores accounts	A customer has accounts.
Composition	`Bank` stores customers	A bank has customers.

Composition is often easier to change because the contained object can be replaced without changing the type hierarchy.

```
class Engine:
```

```
    def __init__(self, horsepower: int):
        self.horsepower = horsepower

class Car:
    def __init__(self, engine: Engine):
        self.engine = engine
```

`Car` is not a subclass of `Engine`; instead, it stores an `Engine` object.

Concept 9: Python Data-Model (Dunder) Methods

A **dunder method** is a method whose name has the form `__name__`. These methods allow user-defined objects to participate naturally in Python syntax.

Method	Enables	Meaning
`__init__`	object construction	Initializes instance state.
`__str__`	`print(obj)`	Human-readable representation.
`__repr__`	`repr(obj)`	Developer-oriented/unambiguous representation.
`__len__`	`len(obj)`	Defines object size.
`__getitem__`	`obj[key]`	Supports indexing/key lookup.
`__contains__`	`x in obj`	Defines membership testing.
`__iter__`	`for x in obj`	Makes the object iterable.
`__bool__`	`if obj:`	Defines truth-value testing.
`__call__`	`obj(...)`	Makes an instance callable.
`__eq__`	`a == b`	Defines equality.
`__add__`	`a + b`	Defines addition for compatible objects.

A container example

```
class Playlist:
    def __init__(self, *songs: str) -> None:
        self._songs = list(songs)

    def __len__(self) -> int:
        return len(self._songs)

    def __getitem__(self, index: int) -> str:
        return self._songs[index]

    def __contains__(self, song: str) -> bool:
        return song in self._songs

    def __iter__(self):
        return iter(self._songs)

    def __bool__(self) -> bool:
        return bool(self._songs)

    def __call__(self, index: int) -> str:
        return f"Now playing: {self._songs[index]}"
```

Now Python can understand expressions such as `len(p)`, `p[0]`, `"Intro" in p`, `for song in p`, `bool(p)`, and `p(1)`.

How `__len__` and `__bool__` interact

If a class has no explicit `__bool__`, Python can use `__len__` to determine truthiness. Therefore, an empty container-like object can naturally evaluate to `False`.

`__repr__` vs `__str__`

`__str__` is intended for readable output, while `__repr__` should be useful for debugging and identification of the object. They serve different audiences.

Concept 10: Encapsulation and Abstraction in the Capstone

Although these concepts were introduced earlier, Problem 05 uses them together, so they are reviewed here.

Encapsulation

Encapsulation means controlling how an object's internal state is accessed or changed. Python commonly uses naming conventions such as `_name` for protected-by-convention attributes and `__name` for name-mangled attributes.

For a bank account, it is safer to prevent arbitrary direct assignments such as `account.__balance = -100000`. Instead, public methods such as `deposit()` and `withdraw()` validate operations and update the internal state.

Abstraction with `ABC`

An abstract base class defines an interface that concrete subclasses must implement.

```
from abc import ABC, abstractmethod

class Account(ABC):
    @abstractmethod
    def available_funds(self) -> float:
        pass

    @abstractmethod
    def monthly_update(self) -> str:
        pass
```

The abstract class describes *what* an account must provide without deciding exactly *how* every account type performs the operation. `SavingsAccount` and `CurrentAccount` can then implement the same interface differently.

> An abstract class containing unimplemented abstract methods cannot normally be instantiated directly. The purpose is to define a contract for concrete subclasses.

Concept 11: Putting the Concepts Together

The five practical problems deliberately form a progression:

Problem	Main focus	What the student should demonstrate
01	Class-level state	Correct class/instance distinction, class methods, static validation, and shadowing.
02	Inheritance hierarchy	Multilevel/hierarchical inheritance, `super()`, overriding, polymorphic payroll.
03	Multiple inheritance	Diamond problem, C3 MRO, cooperative `super()`, capability ordering.
04	Composition + dunders	HAS-A design and native-like container behavior.
05	Integrated system	Abstraction, encapsulation, inheritance, composition, class state, factories, and polymorphism.

Recommended problem-solving order: (1) identify entities, (2) identify their state, (3) identify relationships as IS-A or HAS-A, (4) choose inheritance/composition, (5) define the common interface, (6) implement methods, (7) trace edge cases, and only then (8) write the input/output driver.

Assignment Problems (90-Minute Practical Session)

Problem 01: Campus Student Registry & Class-Level State Engine

Difficulty: Easy
OOP Concepts Tested
Class vs Instance Variables | `@classmethod` Alternative Constructors | `@staticmethod` | Attribute Shadowing
Related Free Practice Problems: LeetCode 1797: Design Authentication Manager | LeetCode 1244: Design A Leaderboard

Real-World Use Case: A university ERP keeps *global* state (institute name, number of enrolled students) that must be shared by every record, while each student keeps private data. Records arrive from different sources (a text row, a key/value map), so several construction paths are required.

Design Specifications

1. Class `Student` with **class variables** `university = "DS_University"` and `student_count = 0`, and instance variables `name` (str), `roll` (str), `cgpa` (float).
2. `__init__` validates the CGPA with `Student.is_valid_cgpa`. An invalid value raises `ValueError` (the driver prints `Error: Invalid CGPA`). A rejected record must **not** increase `student_count`.
3. `@classmethod from_string(cls, text)` parses `"Name-Roll-CGPA"`; `@classmethod from_dict(cls, data)` builds from a dict with keys `name`, `roll`, `cgpa`. Both must return `cls(...)`, not `Student(...)`.
4. `@classmethod change_university(cls, new_name)` updates the shared class variable.
5. `@staticmethod is_valid_cgpa(cgpa) -> bool` ($0.0 \le$ cgpa ≤ 10.0) and `@staticmethod letter_grade(cgp -> str`: $\ge$9 A, $\ge$8 B, $\ge$7 C, $\ge$6 D, otherwise F.
6. `__str__` returns `<name> (<roll>) | CGPA: <c:.2f> | Grade: <g> | University: <uni>`, reading `self.university` so that shadowing is visible.
7. Command `SHADOW roll name` assigns `student.university = name` (an *instance* attribute). That student must keep its own value after later `RENAME_UNI` calls; other students must be unaffected.

Input Format

- Line 1: Integer Q, the number of operations.
- Next Q lines: `ADD name roll cgpa` | `ADD_STR Name-Roll-CGPA` | `ADD_DICT name=X roll=Y cgpa=Z` | `RENAME_UNI name` | `SHADOW roll name` | `INFO roll` | `COUNT` | `GRADE cgpa`
- Output formats: `Student Added: <name> (<roll>)`, `University set to: <name>`, `Instance override set for <roll>`, `Total Students: <n>`, `Grade for <cgpa>: <g>`.

Constraints

- $1 \le Q \le 50$. Roll numbers are unique. Names and university names contain no spaces.
- CGPA is a real number; values outside [0, 10] are invalid records. `INFO` is only issued for existing rolls.

Sample Input 1

```
10
ADD Asha R1 8.5
ADD_STR Ravi-R2-9.2
ADD_DICT name=Meena roll=R3 cgpa=11.0
ADD_DICT name=Kiran roll=R4 cgpa=6.4
RENAME_UNI Global_Tech
SHADOW R2 Legacy_Institute
INFO R1
INFO R2
COUNT
GRADE 7.5
```

Sample Output 1

```
Student Added: Asha (R1)
Student Added: Ravi (R2)
Error: Invalid CGPA
Student Added: Kiran (R4)
University set to: Global_Tech
Instance override set for R2
Asha (R1) | CGPA: 8.50 | Grade: B | University: Global_Tech
Ravi (R2) | CGPA: 9.20 | Grade: A | University: Legacy_Institute
Total Students: 3
Grade for 7.5: C
```

Explanation: Meena has CGPA 11.0, so her record is rejected and the counter stays at 3. After `RENAME_UNI` the shared class variable changes for Asha, but Ravi was given an instance-level override and still prints `Legacy_Institute`.

Viva focus: why `from_string` uses `cls`; what `Student.__dict__` vs `asha.__dict__` contain before and after `SHADOW`; why `letter_grade` is static.

Problem 02: Corporate Payroll Hierarchy (Multilevel & Hierarchical Inheritance)

Difficulty: Easy-Medium
OOP Concepts Tested
Multilevel Inheritance | Hierarchical Inheritance | `super()` Chaining | Polymorphic Dispatch | `isinstance` / MRO
Related Free Practice Problems: LeetCode 1166: Design File System | LeetCode 379: Design Phone Directory

Real-World Use Case: HR systems compute monthly pay for many staff types. Each role changes the pay rule of its parent slightly, and the payroll run must treat every employee uniformly.

Design Specifications

1. Base class `Employee(name, base_salary)` with a class attribute `role = "Employee"`, method `monthly_pay() -> float` (returns base salary), a hook `_extra() -> str` (default empty string) and `describe() -> str` returning `"Name [Role<extra>]"`.
2. **Multilevel chain:** `Manager(Employee)` adds `team_size`; pay = `super().monthly_pay()` + 500 × team_size; `_extra` returns `", team=<n>"`. `Director(Manager)` pay = `super().monthly_pay()` × 1.20 (reuse `Manager`'s `__init__` and `_extra` without rewriting them).
3. **Hierarchical siblings:** `Engineer(Employee)` with `level` (pay = base × (1 + 0.10 × level), `_extra` = `", level=<n>"`); `Intern(Employee)` pay = base × 0.5.
4. Each class overrides the class attribute `role` with its own name.
5. A `Payroll` driver iterates over a heterogeneous list, prints each employee, computes the total and the highest paid person. It must **not** use `isinstance` or type names to compute pay.
6. Query `ISA name ClassName` prints the result of `isinstance`; `MRO ClassName` prints the MRO names joined by `" -> "`.

Input Format

- Line 1: Integer N. Next N lines: `EMPLOYEE name base` | `MANAGER name base team` | `DIRECTOR name base team` | `ENGINEER name base level` | `INTERN name base`
- Next line: Integer Q. Next Q lines: `ISA name ClassName` | `MRO ClassName`
- Output: one line per employee (`<describe> -> Pay: $<x.xx>`), then `Total Monthly Payroll: $<x.xx>`, `Highest Paid: <name> ($<x.xx>)`, then one line per query.

Constraints

- $1 \le N \le 50$, $1000 \le$ base ≤ 100000, $1 \le$ level ≤ 3, team ≥ 0. Names are unique with no spaces. Ties for highest pay: the earliest listed wins.

Sample Input 1

```
5
EMPLOYEE Anil 3000
MANAGER Bhavna 5000 4
DIRECTOR Chetan 8000 10
ENGINEER Divya 4000 3
INTERN Esha 2000
4
ISA Chetan Manager
ISA Esha Manager
ISA Divya Employee
MRO Director
```

Sample Output 1

```
Anil [Employee] -> Pay: $3000.00
Bhavna [Manager, team=4] -> Pay: $7000.00
Chetan [Director, team=10] -> Pay: $15600.00
Divya [Engineer, level=3] -> Pay: $5200.00
Esha [Intern] -> Pay: $1000.00
Total Monthly Payroll: $31800.00
Highest Paid: Chetan ($15600.00)
True
False
True
Director -> Manager -> Employee -> object
```

Explanation: Manager pay = 5000 + 4 × 500 = 7000. Director pay chains through `Manager`: (8000 + 10 × 500) × 1.2 = 15600. Engineer = 4000 × 1.3 = 5200; Intern = 2000 × 0.5 = 1000. A `Director` is a `Manager` (True), an `Intern` is not (False).

Viva focus: trace the `super()` chain for `Director.monthly_pay()`; what happens if `Director` copies the manager formula instead; multilevel vs hierarchical.

Problem 03: Smart-Home Device Fabric (Multiple Inheritance & MRO)

Difficulty: Medium
OOP Concepts Tested
Multiple Inheritance | Diamond Problem | C3 MRO | Cooperative `super()` | MRO Introspection
Related Free Practice Problems: LeetCode 155: Min Stack | LeetCode 1381: Design a Stack With Increment Operation

Real-World Use Case: IoT products combine capabilities: a video doorbell is a camera *and* a speaker; a security hub adds motion sensing. All of them share one core boot routine that must run **exactly once**, however many capabilities are mixed in.

Design Specifications

1. Base `Device(name)` with class attribute `feature = None`, list `log`, and `power_on()` that **resets** the log to `["Device core booted"]`.
2. `Camera(Device)` (feature `video`, log `"Camera sensor online"`), `Speaker(Device)` (`audio`, `"Speaker amplifier online"`), `Sensor(Device)` (`motion`, `"Motion sensor armed"`).
3. `Doorbell(Camera, Speaker)` (feature `chime`, log `"Doorbell chime ready"`) and `SecurityHub(Doorbell, Sensor)` (feature `uplink`, log `"Hub uplink established"`).
4. Every `power_on` must call `super().power_on()` **first**, then append its own message. Direct calls such as `Camera.power_on(self)` are forbidden.
5. `mro_names()` returns the MRO class names joined by `" -> "` (including `object`). `capabilities()` walks `type(self).__mro__`, reads each class's *own* `feature` via `vars(cls)`, and returns them in MRO order, skipping `None`.

Input Format

- Line 1: Integer Q. Next Q lines: `NEW TYPE name` (TYPE ∈ CAMERA, SPEAKER, SENSOR, DOORBELL, HUB) | `POWER_ON name` | `MRO name` | `CAPS name`
- Outputs: `Created <ClassName> '<name>'`; `Boot sequence for <name>:` followed by numbered, 2-space indented log lines; `MRO(<name>): ...`; `Capabilities(<name>): a, b, c`. Unknown type: `Error: Unknown Device Type`. Unknown device name: `Error: Unknown Device`.

Constraints

- $1 \leq Q \leq 60$. Device names are unique with no spaces.

Sample Input 1

```
8
NEW DOORBELL Front
NEW HUB Home
NEW DRONE Sky
POWER_ON Front
POWER_ON Home
MRO Home
CAPS Home
CAPS Ghost
```

Sample Output 1

```
Created Doorbell 'Front'
Created SecurityHub 'Home'
Error: Unknown Device Type
Boot sequence for Front:
  1. Device core booted
  2. Speaker amplifier online
  3. Camera sensor online
  4. Doorbell chime ready
Boot sequence for Home:
  1. Device core booted
  2. Motion sensor armed
  3. Speaker amplifier online
  4. Camera sensor online
  5. Doorbell chime ready
  6. Hub uplink established
MRO(Home): SecurityHub -> Doorbell -> Camera -> Speaker -> Sensor -> Device -> object
Capabilities(Home): uplink, chime, video, audio, motion
Error: Unknown Device
```

Explanation: The hub's MRO is `SecurityHub, Doorbell, Camera, Speaker, Sensor, Device, object`. Because each `power_on` calls `super()` before appending, the boot log is the *reverse* of the MRO: `Device` boots once at the bottom, then Sensor, Speaker, Camera, Doorbell and Hub. Capabilities follow the forward MRO.

Viva focus: why `Speaker` runs between `Camera` and `Device` even though `Camera` does not inherit from it; what breaks (and how many times `Device` boots) if `Camera` calls `Device.power_on(self)`; what MRO Python would compute if the bases of `Doorbell` were swapped.

Problem 04: Smart Shopping Cart: A Native-Feeling Container (Composition & Dunder Methods)

Difficulty: Medium-Hard
OOP Concepts Tested
Composition (HAS-A) | Container Dunders | `__call__` | `__bool__` | Simulated Method Overloading | `__repr__`
Related Free Practice Problems: LeetCode 173: Binary Search Tree Iterator | LeetCode 622: Design Circular Queue

Real-World Use Case: E-commerce back ends expose domain objects that feel like built-in Python types: you can check membership, index, iterate, test emptiness and even "call" a cart to price it with a discount.

Design Specifications

1. Class `Item(name, price, qty=1)` with read-only property `subtotal`, `__eq__` (same name and price), a consistent `__hash__`, and `__repr__` returning `Item('pen', 2.5, 4)`.
2. Class `Cart(owner)` **has a** private list of `Item` objects (composition; do **not** subclass `list`).
3. **Simulated overloading** in `Cart.add(*args)`: `add(item)`, `add(name, price)` (qty 1) and `add(name, price, qty)`. Adding an existing name increases its quantity. Any other argument shape raises `TypeError`.
4. `__len__` returns the number of *distinct* items; `__bool__` is False when the cart is empty; `__contains__(name)`; `__iter__` yields items in insertion order.
5. `__getitem__(key)`: an `int` is a positional index (`IndexError` if out of range), a `str` is an item name (`KeyError` if absent).
6. `__call__(discount_percent=0)` returns the total after discount: sum(subtotals) $\times$ (1 $-$ pct/100).
7. `__repr__` returns `Cart(owner='<owner>', items=<n>)`. `remove(name)` deletes an item or raises `KeyError`.

Input Format

- Line 1: Owner name. Line 2: Integer Q. Next Q lines: `ADD name price [qty]` | `ADD_ITEM name price qty` (builds an `Item` first) | `REMOVE name` | `GET key` | `HAS name` | `LEN` | `EMPTY` | `LIST` | `TOTAL pct` | `REPR`
- For `GET`, an all-digit key is an index, otherwise a name. Outputs: `Added: <name>`; `Removed: <name>`; `repr(item)`; `True/False`; `Distinct Items: <n>`; `Cart Empty: True/False`; `<i>. <name> x<qty> @ $<p> = $<s>`; `Total (<pct:g>% off): $<x.xx>`. Errors: `Error: Item Not Found`, `Error: Index Out of Range`.

Constraints

- $1 \le Q \le 60$, price > 0, qty ≥ 1, $0 \le$ pct ≤ 100. Item names contain no spaces.

Sample Input 1

```
Zara
14
EMPTY
ADD pen 2.50 4
ADD notebook 45 2
ADD_ITEM bag 999.99 1
ADD pen 2.50
LIST
GET 1
GET bag
GET 7
HAS pencil
LEN
REMOVE notebook
TOTAL 10
REPR
```

Sample Output 1

```
Cart Empty: True
Added: pen
Added: notebook
Added: bag
Added: pen
1. pen x5 @ $2.50 = $12.50
2. notebook x2 @ $45.00 = $90.00
3. bag x1 @ $999.99 = $999.99
Item('notebook', 45.0, 2)
Item('bag', 999.99, 1)
Error: Index Out of Range
False
Distinct Items: 3
Removed: notebook
Total (10% off): $911.24
Cart(owner='Zara', items=2)
```

Explanation: The second `ADD pen` merges into the first (qty $4 + 1 = 5$), so the cart has 3 distinct items. `GET 7` triggers `IndexError` from `__getitem__`. `"pencil" in cart` calls `__contains__` and is False. After removing the notebook: $(12.50 + 999.99) \times 0.9 = 911.24$ via `cart(10)`.

Viva focus: why composition beats subclassing `list` here; how `__bool__` and `__len__` interact; what `__hash__` must satisfy when `__eq__` is overridden; how `for x in cart` would behave if only `__getitem__` were defined.

Problem 05: Mini Core-Banking Platform (Capstone)

Difficulty: Hard
OOP Concepts Tested
Composition | `abc.ABC` | Inheritance & Overriding | Class Variables | `@classmethod` / `@staticmethod` | Encapsulation | Polymorphic Batch Processing
Related Free Practice Problems: LeetCode 146: LRU Cache | LeetCode 353: Design Snake Game

Real-World Use Case: A bank owns customers, customers own accounts, and each account type applies its own overdraft and month-end rules. Bank-wide state (its name, a global transaction counter) is shared, while account balances must be protected from direct tampering.

Design Specifications

1. Abstract class `Account(ABC)` with a **private** balance (`__balance`) exposed through a **read-only** `balance` property, a protected mutator `_apply(delta)`, a class attribute `kind`, and abstract methods `available_funds() -> float` and `monthly_update() -> str`. `Account(...)` must raise `TypeError`.
2. `SavingsAccount`: available = balance; class constant `MONTHLY_RATE = 0.01`; month-end adds interest and returns `"<id> (Savings): +$<i> interest -> $<bal>"`.
3. `CurrentAccount`: `OVERDRAFT_LIMIT = 500.0`, `MONTHLY_FEE = 10.0`; available = balance + overdraft; month-end deducts the fee and returns `"<id> (Current): -$10.00 fee -> <bal>"`.
4. `Customer(customer_id, name)` **has** a dict of accounts (composition). `Bank` has customers and a flat account lookup; class variables `bank_name` and `total_transactions`; `@classmethod rename(cls, new_name)`; `@staticmethod valid_amount(x)` ($x > 0$); and a registry dict mapping `"SAVINGS"`/`"CURRENT"` to their classes (a factory; no `if/elif` on type names).
5. `deposit`, `withdraw` (allowed only if amount $\leq$ `available_funds()`), and `transfer` (atomic: on any failure nothing changes). Each *successful* deposit, withdrawal or transfer increments `total_transactions` by 1 (opening balances and month-end postings do not count). Check order: account exists, then amount valid, then funds.
6. `MONTH_END` calls `monthly_update()` on every account in opening order without checking types. Money is formatted `$x.xx`, and negatives as `-$x.xx`.

Input Format

- Line 1: Bank name. Line 2: Integer K, then K lines `CustomerID Name`. Next line: Integer Q. Next Q lines: `OPEN cid accid SAVINGS|CURRENT initial` | `DEPOSIT accid amt` | `WITHDRAW accid amt` | `TRANSFER from to amt` | `MONTH_END` | `RENAME name` | `SUMMARY cid` | `STATS`
- Outputs: `Opened <Kind> account <id> for <Name> with $<x>`; `Deposited $<x> to <id>. Balance: $<b>`; `Withdrew $<x> from <id>. Balance: $<b>`; `Transferred $<x> from <a> to <b>`; `Bank renamed to: <name>`; the `SUMMARY` block and `Bank: <name> | Transactions: <n>`. Errors: `Error: Insufficient Funds`, `Error: Invalid Amount`, `Error: Account Not Found`.

Constraints

- $1 \leq K \leq 20$, $1 \leq Q \leq 100$. IDs and names contain no spaces and are unique. Initial balances are ≥ 0.

Sample Input 1

```
NovaBank
2
C01 Alice
C02 Bob
13
OPEN C01 S1 SAVINGS 1000
OPEN C01 K1 CURRENT 200
OPEN C02 S2 SAVINGS 500
DEPOSIT S1 250
WITHDRAW K1 600
WITHDRAW K1 200
TRANSFER S1 K1 300
TRANSFER S2 S1 800
MONTH_END
RENAME NovaTrust
SUMMARY C01
STATS
DEPOSIT S9 50
```

Sample Output 1

```
Opened Savings account S1 for Alice with $1000.00
Opened Current account K1 for Alice with $200.00
Opened Savings account S2 for Bob with $500.00
Deposited $250.00 to S1. Balance: $1250.00
Withdrew $600.00 from K1. Balance: -$400.00
Error: Insufficient Funds
Transferred $300.00 from S1 to K1
Error: Insufficient Funds
S1 (Savings): +$9.50 interest -> $959.50
K1 (Current): -$10.00 fee -> -$110.00
S2 (Savings): +$5.00 interest -> $505.00
Bank renamed to: NovaTrust
Customer Alice (C01): 2 account(s)
  S1 Savings: $959.50
  K1 Current: -$110.00
  Net Worth: $849.50
Bank: NovaTrust | Transactions: 3
Error: Account Not Found
```

Explanation: K1 starts at \$200 with a \$500 overdraft, so withdrawing \$600 (available \$700) succeeds and leaves −\$400; a further \$200 exceeds the remaining \$100 of available funds and is rejected. Transferring \$300 from S1 succeeds. The \$800 transfer from S2 (balance \$500) fails atomically. Month-end applies 1% interest to S1 and S2 and the \$10 fee to K1 through one polymorphic call. Only 3 operations succeeded (deposit, withdrawal, transfer), so the counter is 3.

Viva focus: why `_apply` exists instead of a balance setter; what happens to a second `Bank` instance after `rename` (class variable); how you would add a `StudentAccount` without editing `Bank`; inheritance vs composition for each relationship in the model.